

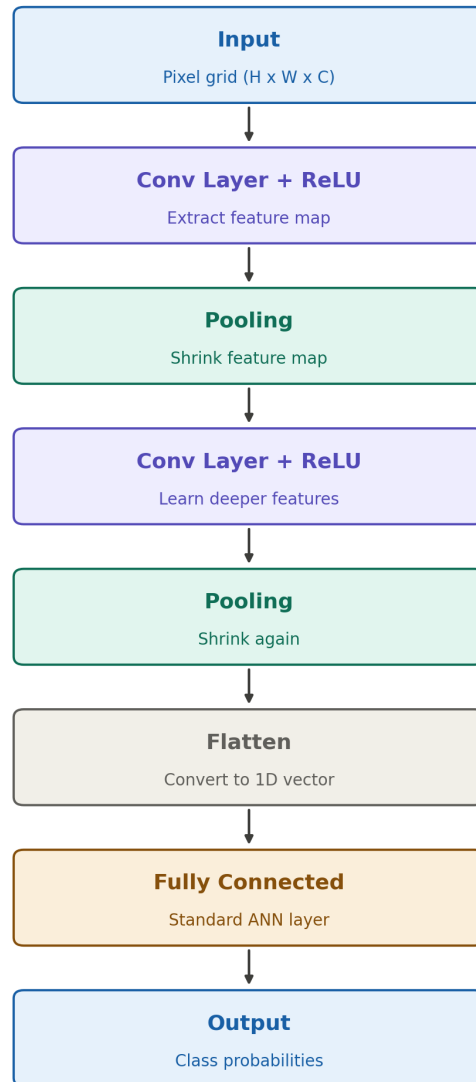
# CNN — Complete Notes

Convolutional Neural Networks: architecture, math, and worked examples

---

## 1. Architecture — Full Pipeline

### CNN Architecture — Full Pipeline



*Input → Conv layer + ReLU (extract feature map) → Pooling (shrink feature map) → Conv layer + ReLU (learn deeper features) → Pooling (shrink again) → Flatten (convert to 1D vector) → Fully Connected (ANN) → Output*

**Preprocessing note (outside the CNN):** if we start from a custom image file (.jpg/.png), we first Decode → Resize → Normalize it into a fixed-size pixel array. This is data preparation, not a learned layer. If the data is already a pixel array (e.g. MNIST/CIFAR), this step is skipped.

**One-line summary:** CNN = feature extractor (conv + pool, repeated) followed by an ANN (fully connected layer) at the end.

## 2. Image as a Matrix

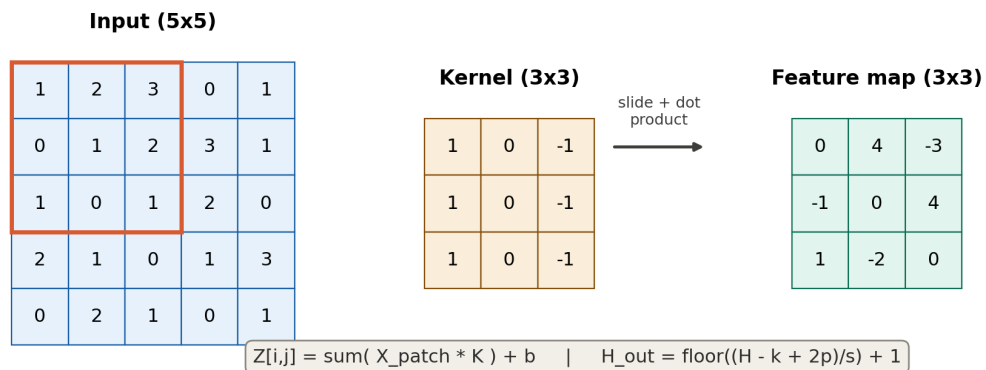
An image is literally a matrix of numbers — nothing more.

| Type      | Shape                 | Meaning                                  |
|-----------|-----------------------|------------------------------------------|
| Grayscale | $H \times W$          | one number per pixel (brightness, 0-255) |
| RGB       | $H \times W \times 3$ | three stacked grids (Red, Green, Blue)   |

Example: a 21×21 RGB image is a 21×21×3 matrix. The size is either read from the file (`img.size` in PIL, `img.shape` in OpenCV) or set deliberately by resizing the image before feeding it to the network — CNNs require a fixed, consistent input size.

## 3. Convolution Layer

### Convolution Operation



### 3.1 What it does

A small matrix of learnable weights, called a **kernel** (or **filter**), slides over the input and computes a local weighted sum at every position. The result is a **feature map**: a grid showing where and how strongly a specific pattern was detected.

- Early conv layers detect simple patterns (edges, corners)
- Deeper conv layers detect complex patterns (textures, parts, objects), because they operate on the previous layer's feature maps, not raw pixels

### 3.2 Forward pass formula

At each valid position (i, j):

$$Z[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i+m, j+n] \cdot K[m, n] + b$$

| Symbol | Meaning                                              |
|--------|------------------------------------------------------|
| X      | input matrix                                         |
| K      | kernel (k × k)                                       |
| b      | bias (scalar)                                        |
| Z[i,j] | one value of the output feature map (pre-activation) |

### 3.3 Output size formula

$$H_{out} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1$$

| Symbol  | Meaning                                |
|---------|----------------------------------------|
| H       | input height / width (pixels)          |
| k       | kernel size                            |
| p       | padding (added to EACH side)           |
| s       | stride                                 |
| floor() | round down to the nearest whole number |

**This formula gives SIZE only** — it tells you the shape of the output, never the actual numbers inside it. The numbers come only from applying the formula in §3.2 at every position.

### 3.4 Worked example — 21×21 input, 3×3 kernel, stride 1, no padding

$$H_{out} = \left\lfloor \frac{21 - 3 + 2(0)}{1} \right\rfloor + 1 = \left\lfloor \frac{18}{1} \right\rfloor + 1 = 19$$

So a 21×21 input with these settings produces a **19×19** feature map. (Intuition: valid kernel start positions are 0,1,...,18 — 19 of them — in each direction.)

If the conv layer uses C filters, the full output volume is 19×19×C — the spatial size never changes with filter count, only the depth does.

### 3.5 Where floor() actually matters (messy example)

H=21, k=4, s=2, p=0:

$$H_{out} = \left\lfloor \frac{21 - 4 + 0}{2} \right\rfloor + 1 = \lfloor 8.5 \rfloor + 1 = 9$$

The kernel cannot take a "half step" — leftover edge pixels are simply unused.

## 4. Kernel / Filter

- A small matrix of **learnable** weights that slides over the input.
- Size ( $k \times k$ ) is a **hyperparameter** — chosen before training, not learned.
- — Small kernel ( $3 \times 3$ ): captures fine local detail, needs more layers to build up a big-picture view.
- — Large kernel ( $5 \times 5$ ,  $7 \times 7$ ): captures a bigger pattern in one step, at higher compute cost.
- The **values inside** the kernel start random and are learned via backpropagation.
- One kernel detects **one** pattern → one feature map. A conv layer with C kernels produces C feature maps (stacked as channels).

## 5. Stride (s)

How many pixels the kernel jumps per step.

| Stride | Effect                                                            |
|--------|-------------------------------------------------------------------|
| s = 1  | moves 1 px at a time → large output, high overlap between windows |
| s = 2  | moves 2 px at a time → output shrinks faster, less overlap        |

## 6. Padding (p)

Extra border (usually zeros) added around the input before the kernel slides over it.

**Why it's needed:**

- Without padding, output size always shrinks ( $H_{out} < H$ ).
- Border pixels are touched by the kernel far fewer times than center pixels, so information at the edges is under-represented.
- Padding lets us control the output size (e.g. "same" padding keeps  $H_{out} = H$ ) and gives border pixels fairer treatment.

## 7. ReLU (Activation Function)

Applied immediately after the conv layer's raw output (Z), never on raw pixels.

Raw pixels  $\rightarrow$  Convolution ( $Z = X \blacksquare K + b$ )  $\rightarrow$  ReLU  $\rightarrow$  a (final conv layer output)

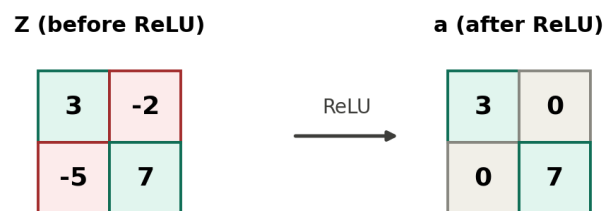
Raw pixel values are usually already non-negative (0-255), so there's nothing for ReLU to "fix" there. Negative values only appear after convolution, because kernel weights can be negative.

## 7.1 Formula

$$a[i, j] = \max(0, Z[i, j])$$

Negative  $\rightarrow$  becomes 0. Positive  $\rightarrow$  unchanged. **Shape never changes** — ReLU only affects values, never size (size is fixed by the  $H_{\text{out}}$  formula).

## 7.2 Worked example



## 7.3 Why ReLU is necessary (not optional)

Without any activation function, stacking conv layers collapses mathematically into a **single linear layer**, no matter how many layers are stacked:

$$z_2 = w_2(w_1x + b_1) + b_2 = (w_2w_1)x + (w_2b_1 + b_2) = Wx + B$$

Same form as one layer — depth adds nothing. Real-world visual patterns (edges, curves, textures, objects) are **non-linear**, and a purely linear stack can only ever draw straight-line decision boundaries. ReLU breaks the linear chain, so each additional layer genuinely adds representational power.

It also has practical benefits: cheap to compute (no exponentials, unlike sigmoid/tanh), and its clean 0/1 gradient reduces the vanishing-gradient problem during backpropagation.

## 8. Pooling Layer

### Pooling Operation (Max Pooling)

Input feature map (4x4)

|   |   |   |   |
|---|---|---|---|
| 0 | 2 | 5 | 3 |
| 3 | 0 | 0 | 1 |
| 1 | 0 | 7 | 0 |
| 2 | 4 | 6 | 0 |

max pooling  
f=2, s=2



Output (2x2)

|   |   |
|---|---|
| 3 | 5 |
| 4 | 7 |

$$H_{out} = \text{floor}((H - F)/S) + 1 \quad | \quad \text{each box} \rightarrow \text{max(box)} \text{ or avg(box)}$$

### 8.1 What it does

Shrinks the feature map, keeping only the strongest (or average) signal from each local region. Input to pooling = output of conv layer + ReLU.

### 8.2 Output size formula

$$H_{out} = \left\lfloor \frac{H - f}{s} \right\rfloor + 1$$

| Symbol | Meaning          |
|--------|------------------|
| H      | input size       |
| f      | pool window size |
| s      | stride           |

Note: no padding term — pooling almost never uses padding. Same principle as convolution: **this formula gives SIZE only**. The actual output values require doing the real max/avg operation on every window.

### 8.3 Worked example — 4x4 input, f=2, s=2

$$H_{out} = \left\lfloor \frac{4 - 2}{2} \right\rfloor + 1 = 2$$

## 8.4 Why $f = s = 2$ is the standard default

When stride equals pool size, windows **tile with zero overlap and zero gaps** — every pixel is used exactly once, giving a clean 50% shrink each time. Smaller shrink steps ( $2 \times 2$  rather than a large window) also preserve more information gradually, letting conv layers extract useful patterns at each intermediate scale rather than losing most detail in one aggressive squish.

## 8.5 Max Pooling vs Average Pooling

Both use the exact same window/stride mechanics — the only difference is the operation applied inside each window.

$$\text{Max pooling: } a_{out} = \max(\text{window}) \quad \text{Avg pooling: } a_{out} = \frac{1}{f^2} \sum(\text{window})$$

| Window       | Values    | Max | Avg  |
|--------------|-----------|-----|------|
| top-left     | {5,3,2,9} | 9   | 4.75 |
| top-right    | {1,8,4,6} | 8   | 4.75 |
| bottom-left  | {7,1,2,8} | 8   | 4.5  |
| bottom-right | {0,3,5,1} | 5   | 2.25 |

**Max pooling is the standard default** in almost every modern CNN (VGG, ResNet, etc.) for the conv+pool blocks in the middle of the network. **Average pooling** is more of a specialized/final-stage tool — e.g. Global Average Pooling, used right before the output layer in some architectures.

## 9. How Many Conv+Pool Blocks to Stack?

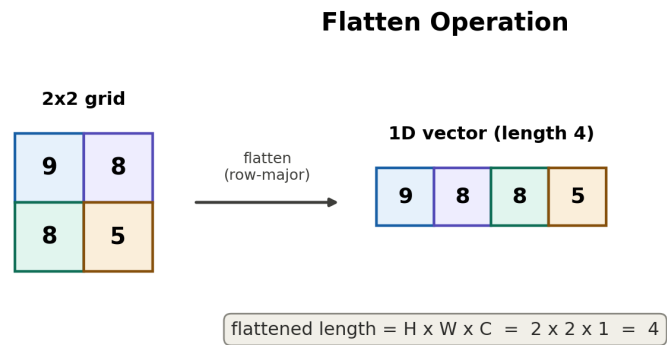
There is no fixed rule — this is a design choice (hyperparameter), guided by:

- **Input size** — each pooling round roughly halves the spatial size, so a small image can only support a few rounds before running out of pixels.
- **Task complexity** — simple tasks (digit recognition) need only 2-3 blocks; complex tasks (real-world photos, many classes) typically need 5+.
- **Known reference architectures** — most practitioners start from a proven design rather than deriving depth from scratch:

| Architecture   | Conv+Pool blocks (roughly)  |
|----------------|-----------------------------|
| LeNet (simple) | 2                           |
| VGG-16         | 5                           |
| ResNet-50      | many (uses residual blocks) |

More blocks → deeper → more abstract features, but higher compute cost and risk of vanishing gradients.  
Fewer blocks → faster, but may underfit hard tasks.

## 10. Flatten



### 10.1 What it does

Converts the final pooled output (which is 3-dimensional: height  $\times$  width  $\times$  channels) into a single 1D vector, so it can feed into a fully connected (ANN) layer. **No learning happens here — it is purely a reshape.**

### 10.2 Formula

$$\text{flattened length} = H \times W \times C$$

### 10.3 Worked example

Take the max-pooling output from §8.5, [9 8 / 8 5] — a 2x2 grid with C=1 channel:

$$\text{flattened length} = 2 \times 2 \times 1 = 4$$

[9 8 / 8 5]  $\rightarrow$  flatten (row-major)  $\rightarrow$  [9, 8, 8, 5]. Read row by row, left to right, top to bottom.

## 11. Channels (C)

The meaning of "channel" depends on where you are in the network.

| Stage                | What C means                                                           |
|----------------------|------------------------------------------------------------------------|
| Raw input image      | Color format — 1 = grayscale, 3 = RGB, 4 = RGBA (rare)                 |
| After any conv layer | Number of filters/kernels used in that layer (no longer tied to color) |

| Type      | Channels | Typical use                                  |
|-----------|----------|----------------------------------------------|
| Grayscale | 1        | color doesn't matter (simple/medical images) |
| RGB       | 3        | almost all real-world photos (default)       |
| RGBA      | 4        | rare, only when transparency matters         |

**Common convention inside the network:** start with 32 or 64 filters in the first conv layer, then roughly double the filter count each time the spatial size is halved (e.g.  $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ ), so the network



extracts more distinct features per location as spatial detail shrinks.

## 12. Feature Map

A **feature map** is the output of one kernel applied across the whole input — a grid of numbers with the same spatial layout as the input, where each value answers: "was my pattern found here, and how strongly?"

- Large value → the pattern was strongly detected at that position
- Small / negative value → the pattern was not present there

One kernel → one feature map. A conv layer with  $C$  kernels produces  $C$  feature maps stacked together (this stack is exactly the "channels" from §11 after a conv layer).

**Pattern** = a specific visual structure the kernel's weights are tuned to react to — an edge, curve, texture, or (in deeper layers) a shape built from combinations of earlier features. In a trained network, kernels start random and their patterns are discovered automatically through backpropagation, not hand-designed.

## Quick Recap

| Term                          | One-liner                                                                    |
|-------------------------------|------------------------------------------------------------------------------|
| Image                         | a matrix of pixel numbers ( $H \times W$ , or $H \times W \times 3$ for RGB) |
| Convolution                   | slides a kernel over the input, extracts features via dot product            |
| Kernel                        | small learnable matrix; size is a hyperparameter, values are learned         |
| Stride                        | how many pixels the kernel/window jumps per step                             |
| Padding                       | border added around input to control output size and treat edges fairly      |
| $H_{\text{out}}(\text{conv})$ | $\text{floor}((H-k+2p)/s) + 1$ — output SIZE only                            |
| ReLU                          | $\max(0, Z)$ — kills negatives, adds non-linearity so depth matters          |
| Pooling                       | shrinks the feature map, keeping only the strongest/average signal           |
| $H_{\text{out}}(\text{pool})$ | $\text{floor}((H-f)/s) + 1$ — output SIZE only                               |
| Max pooling                   | keeps the strongest value per window — standard default                      |
| Avg pooling                   | keeps the average value per window — specialized/final-stage tool            |
| Conv+Pool repeat count        | design choice, limited by input size and task complexity                     |
| Flatten                       | $H \times W \times C \rightarrow 1D$ vector; reshape only, no learning       |
| Channel ( $C$ )               | color info at input, or filter count after a conv layer                      |
| Feature map                   | grid of "how strongly was this pattern found here" scores                    |
| Pattern                       | a specific visual structure a kernel is tuned to detect                      |